Reducing False Positives in Heuristic DNS Tunneling Detection: A Structural Analysis of Legitimate High-Entropy DNS Names

October 7, 2026 · MSc. Medardo Jairo Suntaxi Cocanguilla, C|EH

Abstract

In September 2026 a tunneling detection module running on a production recursive DNS service blocked the parent domain of an institutional customer after more than a year of intermittent detections on its DKIM records. The review of that case showed that the problem was not limited to one domain. More than twenty other institutional domains had DKIM or Active Directory names classified under the same tunnel pattern, and reverse-mapping (PTR) queries were affected for the same reason.

This paper describes what was observed, why these names trigger a detector based on name length, label count and entropy, and which mitigation was chosen. The short answer is that their own standards make them look like tunnels: a DKIM name delegated to a cloud mail provider reaches 7 labels, an Active Directory GUID record carries 128 random bits in hexadecimal, and every IPv6 PTR query has 34 labels.

Raising the global thresholds was evaluated and discarded, because it reduces false positives only by opening a band in which a real tunnel would go unnoticed. The approach adopted keeps the thresholds as they are and handles exceptions through a verified, narrowly scoped allowlist. All operational data in this paper are anonymized.

1. Context

The environment is a production recursive DNS service. Resolution is spread across several cache nodes that serve different groups of clients. Each node runs a real-time tunneling detection module that inspects query names before the resolver answers.

DNS tunneling (MITRE ATT&CK T1071.004, with exfiltration under T1048) works by encoding data into subdomain labels of a domain whose authoritative server the attacker controls. Tools such as iodine, dnscat2 or DNSExfiltrator produce names like `<encoded-chunk>.<session-id>.t.attacker.example`, and every chunk is a new name that cannot be answered from cache. Since RFC 1035 limits a label to 63 octets and a name to 255, a tunnel tends to use long names with many labels.

The detection module follows a two-stage logic that is common in this type of product. A first filter selects names that exceed a configured length or a configured number of labels. Domains that pass the filter receive a confidence score that grows as new, different names are observed under the same parent domain. When the score crosses the blocking threshold, the parent domain is classified as a tunnel and the resolver denies every query under it, not only the suspicious names.

That last point matters operationally. A block is applied to the whole parent domain, so a false positive on a DKIM subdomain can end up denying the organization's mail host and web services. The configured threshold values are not disclosed in this paper.

2. What was observed

The case started with the block of an institutional customer's parent domain in September 2026. All names in this section are generic: the customer's domain appears as `org-a.example` and client addresses are omitted.

The first check was the detection module's own log on the affected cache node:

```
$ sudo grep -Ei "org-a" <detection-log> | tail -10
```

The history it returned explains most of the case. The DKIM subdomain of the institution had been oscillating between report, block and acquittal for more than a year before the parent domain itself was classified:

Month	Decision	Name evaluated
2025-08	REPORT	`_domainkey.org-a.example`
2025-09	BLOCK	`_domainkey.org-a.example`
2025-11	REPORT	`_domainkey.org-a.example`
2025-11	BLOCK	`_domainkey.org-a.example`
2025-12	ACQUIT	`<UUID-selector>._domainkey.org-a.example`
2026-01	ACQUIT	`<UUID-selector>._domainkey.org-a.example`
2026-03	REPORT	`_domainkey.org-a.example`
2026-03	BLOCK	`_domainkey.org-a.example`
2026-08	REPORT	`org-a.example`
2026-09	BLOCK	`org-a.example`

Two details in this history are worth noting. The DKIM selector used by the institution was a 36-character UUID (version 1, time-based format), so every DKIM lookup carried a long hexadecimal label by design. And the final block was not on the DKIM subtree but on the parent domain, which is the scope that takes everything else down with it.

The resolver's deny log confirmed the impact. The excerpt below, sanitized, covers just over one second of traffic a few days after the block:

```
$ sudo grep -Ei "org-a.*tunnel" /var/log/messages | tail
16:27:06 deny <client-1> app-a.org-a.example.        AAAA
16:27:06 deny <client-2> app-a-dev.org-a.example.    TYPE65
16:27:07 deny <client-3> test-login.org-a.example.   TYPE65
16:27:07 deny <client-3> test-login.org-a.example.   A
16:27:07 deny <client-4> http://mail.org-a.example.  A
16:27:07 deny <client-4> http://mail.org-a.example.  TYPE65
```

In the validation of these entries, ordinary host names were being denied: application front ends, a test login host and the mail server. Several clients were affected at the same moment, and the denied types were A, AAAA and TYPE65 (HTTPS/SVCB, RFC 9460), which current browsers request together with the address records.

Two entries need a comment. The test login host suggests the institution was running application tests around the time of the block, which would have produced a burst of new names under the domain. This could not be confirmed with the institution, so it is treated as a probable contributing factor, not as a cause. The queries for `http://mail...` are malformed: a client is sending a URL as the query name. They are a misconfiguration on the client side and add more distinct names to the domain's count.

With the first case understood, the review was extended to other domains in the same classification. More than twenty other institutional domains had `_domainkey` or `_msdcs` names detected under the same tunnel pattern. Enterprise customers showed the same behavior with Active Directory locator records and with DKIM names delegated to a cloud mail provider. Reverse-mapping queries under `in-addr.arpa` and `ip6.arpa` also appeared among the detections.

3. Why these names look like tunnels

The detected names were grouped by protocol role and measured against the formats defined in their standards. The goal was to separate what depends on this environment from what any resolver with the same type of detector will see. The table uses anonymized names built on the exact templates of each standard; length is counted without the trailing dot.

Family	Anonymized example	Labels	Length	Defined by
DKIM delegated to cloud mail	`selector1-org-a-example._domainkey.tenant-a.x-v1.dkim.mailprovider.example`	7	74	RFC 6376 + provider CNAME
DKIM with UUID selector	`<UUID>._domainkey.org-a.example`	4	61	RFC 6376
AD domain-controller locator	`_ldap._tcp.dc._msdcs.example.com`	6	32	RFC 2782, AD DC Locator
AD site-specific locator	`_ldap._tcp.Default-First-Site-Name._sites.dc._msdcs.example.com`	8	63	RFC 2782, AD DC Locator
AD DSA GUID alias	`<GUID>._msdcs.example.com`	4	55	AD replication
IPv4 reverse mapping	`4.3.2.1.in-addr.arpa`	6	up to 28	RFC 1035
IPv6 reverse mapping	`<32 nibbles>.ip6.arpa`	34	72	RFC 3596

DKIM. RFC 6376 publishes keys at `<selector>._domainkey.<domain>`. When signing is delegated to a cloud mail provider, the customer publishes a CNAME whose target embeds the customer's domain and tenant name. The longest DKIM name found in the logs was 68 characters with 7 labels, which matches that template. Selectors are usually set by the mail platform rather than by the domain owner; in the case described in section 2 the selector was a UUID, so the name carried 128 bits of identifier in hexadecimal on every lookup.

Active Directory. Domain controllers are located through SRV records (RFC 2782) under `_msdcs`. The records seen in the logs had 6 labels for the basic locator and 8 for the site-specific one. Replication aliases use the DSA GUID as the first label; the ones observed were 53 characters long. From an entropy point of view a GUID is a random 128-bit value written in hex, which is exactly what a hex-encoded tunnel payload looks like. A large forest also generates many different GUID names under the same parent, which feeds the uniqueness counter.

Reverse mapping. This family does not need field data to explain it. An IPv4 PTR name always has 6 labels, and an IPv6 PTR name always has 34 labels and 72 characters, because RFC 3596 writes each of the 32 nibbles of the address as a separate label. Any label filter below 6 selects every IPv4 PTR query, and no reasonable label filter excludes IPv6 PTR. Every new address is also a new name under `ip6.arpa`.

Accumulation. None of these families is blocked by a single query. The history in section 2 shows the expected pattern for a confidence model: reports, blocks that expire or are acquitted, and new detections months later, until the parent domain itself crosses the threshold. Bursts of new names, such as the probable application tests, push the score up faster.

It was also checked whether the TYPE65 queries could be contributing. Browsers now send HTTPS/SVCB queries together with A and AAAA, so each host name generates up to three queries. If the module counts observations rather than distinct names, this would raise the score without any change in user behavior. The logs do not show how the module weighs record types, so this remains a hypothesis (see section 5).

4. Options evaluated and decision

Four ways of stopping the false positives were on the table. Each was judged by one criterion: how much detection capability it gives up in exchange for fewer false positives.

Option	Effect on false positives	Cost in detection	Decision
Raise global length and label thresholds	Partial: IPv6 PTR and 7–8 label AD/DKIM names still pass most values	A tunnel built just under the new limits is no longer inspected	Discarded
Allowlist an entire public suffix (for example, an entire second-level suffix)	Removes most cases at once	Every current and future domain under the suffix is exempt, including compromised or newly registered ones	Discarded
Allow specific record types globally	Depends on the type	Opens a channel that a tunnel can use by switching record type	Discarded
Allowlist verified domains or protocol subtrees, one by one	Removes the verified cases	Exemption limited to names whose owner and purpose were checked	Adopted

The threshold option was analyzed in more detail because it is the first one an operator usually tries. Section 3 shows why it does not work here: the families involved go from 4 to 34 labels and up to 74 characters, so any value high enough to exclude them would leave a band in which a tunnel is not inspected at all. Raising the thresholds would stop flagging legitimate cloud and directory services, and in the same move it would stop flagging any tunnel built to stay below the new limits.

The suffix-wide allowlist is the quickest fix and was rejected. A shared suffix covers a large number of organizations with very different security levels, and an attacker who compromises or registers one domain under it would get a channel that the detector never inspects.

The adopted approach keeps the thresholds unchanged and adds exceptions only for domains or subtrees that were verified: registration and ownership, authoritative name servers, and a name structure consistent with the protocol it claims to be. Exceptions are added gradually, one domain at a time, as each one is checked.

Reverse-mapping zones need a separate comment. Adding `in-addr.arpa` and `ip6.arpa` to the allowlist is often treated as risk-free, on the grounds that tunnels cannot be built on them. The risk is low but not zero: whoever obtains reverse delegation for an address block controls an authoritative subtree. Under a delegated IPv6 /48, 20 nibble labels (80 bits) remain free in each query, and the PTR answer can return an arbitrary host name. The bandwidth is small and setting it up requires a real address allocation, so allowlisting these zones was accepted, with the condition of monitoring query volume per reverse subtree.

5. Results and open points

The root cause was established and documented: the blocks were produced by the structure of standard DKIM, Active Directory and PTR names interacting with a two-stage confidence model, not by tunnel activity. Nothing in the logs reviewed pointed to real exfiltration under the affected domains. As a mitigation plan, verified, domain-by-domain allowlisting was defined as the procedure for handling these cases.

No before-and-after metric of false positives is presented here. The allowlist is applied progressively, as each domain is verified, so a meaningful comparison needs several weeks of detection logs after the changes.

Three points remained open at the time of writing:

- **Weight of TYPE65 queries.** It is still to be validated whether the module counts HTTPS/SVCB queries as separate observations. A controlled test, querying one host name with and without TYPE65 from a lab client, would answer it.

- **Effectiveness of each exception.** Each allowlisted domain needs to be checked against later logs, to confirm it is no longer reported and that its query volume stays within its usual range.
- **Residual risk of reverse zones.** Volume per reverse subtree has to be monitored to detect abnormal use of a delegated block, as discussed in section 4.

6. Conclusions

A detector that relies on length, label count and entropy will keep flagging DKIM, Active Directory and PTR names, because their standards produce long, many-labeled and random-looking names. This is a property of the protocols, not of one deployment, and threshold tuning cannot fix it: the values that would exclude these families would also exclude tunnels.

The case also showed that the damage of a false positive depends on the block scope. A detection that started on a DKIM subtree ended up denying the institution's mail and application hosts once the parent domain was classified. With a confidence model, this happened more than a year after the first report, which makes the incident hard to link to its origin unless the detection history is reviewed.

Keeping the thresholds and treating each exception as a controlled change, verified, scoped to the smallest stable domain, and monitored, was the only option that solved the cases without giving up detection. Exceptions are not free: every allowlisted name is a name the detector no longer inspects, and that includes the reverse-mapping zones.

7. Recommendations

For operators of resolvers with this type of detection:

1. **Review the full detection history before acting on a block.** A single grep over the module's log for the parent domain shows whether it is a new detection or the end of months of accumulation, and which subtree started it.
2. **Scope the exception to the protocol subtree when the module allows it.** Allowlisting `_domainkey.<domain>` or `_msdcs.<domain>` keeps the rest of the domain under inspection; allowlisting the parent domain should be the fallback, not the default.
3. **Never allowlist a public suffix.** Each organization under it gets verified separately.
4. **Register every exception as a change:** domain, owner, evidence of legitimacy, case reference and a review date. A six-month review is reasonable for domains whose mail or directory setup can change.
5. **Watch for blocks on customer parent domains instead of waiting for complaints.** A daily check is enough, for example:

```
$ grep "^$(date +%F)" <detection-log> | awk '$3=="BLOCK"{print $4}' | sort | uniq -c | sort -rn
```

6. **Report client-side anomalies to the customer.** Queries carrying URLs as names (`http://...`) are a client misconfiguration and add noise to the detector.
7. **Close the TYPE65 question with a lab test** before deciding whether HTTPS/SVCB traffic needs special handling.
8. **Monitor query volume per reverse subtree** once `in-addr.arpa` and `ip6.arpa` are allowlisted.

For detection engines, the case points to two design changes. The first is to recognize standard protocol structures (`_domainkey`, `_msdcs`, `_sites`, reverse zones) as features of their own instead of scoring them only by length and entropy. The second is to offer a report-only stage

for parent domains above a given size or criticality, so that a block on a large customer domain is reviewed by an operator before it is enforced.

References

1 P. Mockapetris, RFC 1035: Domain Names – Implementation and Specification, IETF, 1987.

2 A. Gulbrandsen, P. Vixie, L. Esibov, RFC 2782: A DNS RR for Specifying the Location of Services (DNS SRV), IETF, 2000.

3 D. Eastlake, A. Panitz, RFC 2606: Reserved Top Level DNS Names, IETF, 1999.

4 S. Thomson et al., RFC 3596: DNS Extensions to Support IP Version 6, IETF, 2003.

5 D. Crocker, T. Hansen, M. Kucherawy, RFC 6376: DomainKeys Identified Mail (DKIM) Signatures, IETF, 2011.

6 B. Schwartz, M. Bishop, E. Nygren, RFC 9460: Service Binding and Parameter Specification via the DNS (SVCB and HTTPS RRs), IETF, 2023.

7 Microsoft, Active Directory Domain Controller Locator and `_msdcs` SRV record documentation.

8 MITRE ATT&CK, T1071.004 Application Layer Protocol: DNS; T1048 Exfiltration Over Alternative Protocol.

9 Open-source DNS tunneling tools: iodine, dnscat2, DNSExfiltrator (project documentation).

Author note

Author: MSc. Medardo Jairo Suntaxi Cocanguilla, Certified Ethical Hacker (C|EH), Ecuador.

Version: 1.0 – October 2026.

The observations come from a production DNS environment the author knows first-hand. The analysis and this paper are the author's own study and do not represent the position of any organization. Domain names, client addresses, host names, configuration values and case references have been anonymized or removed.